Cloud Computing TASK – 5

1. **Pick any one app you use daily (like Zomato, BookMyShow, or Spotify) and write a short description of a new feature you would want to build for it that would require cloud hosting and scaling.**

   Ans :
   App: Spotify

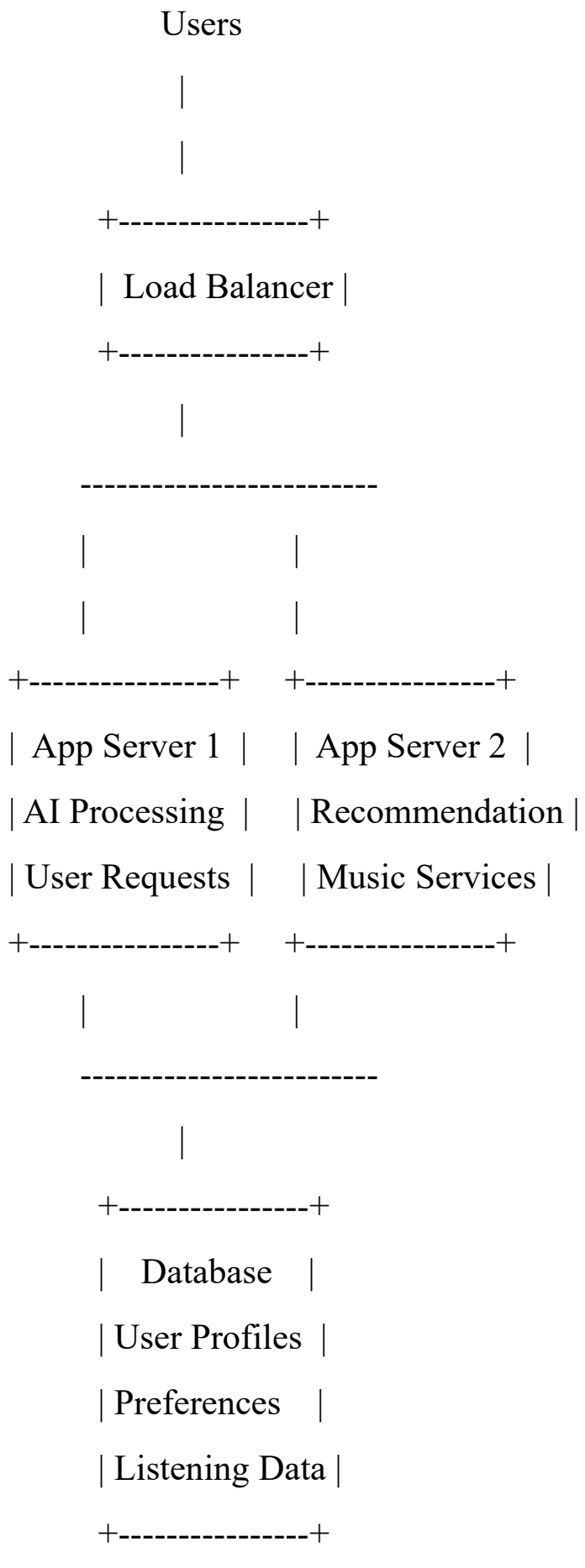
   New Feature: AI-Based Personalized Music Room

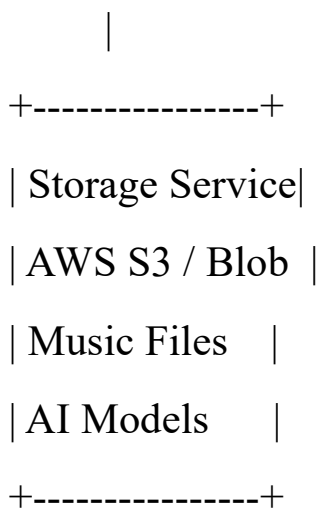
   I would like to build a new feature for Spotify called AI-Based Personalized Music Room. This feature would create a customized virtual music space for each user based on their listening history, mood, activity, and preferences. Users could select activities like studying, exercising, relaxing, or travelling, and the AI would automatically generate playlists, recommend podcasts, and adjust music choices in real time.

   This feature would require cloud hosting and scaling because millions of users may access personalized recommendations simultaneously. Cloud servers would handle large amounts of user data, AI processing, and music requests. Auto scaling would help increase server capacity during peak usage times and reduce resources when demand is lower, ensuring smooth performance while controlling costs.

2. **Draw a basic cloud architecture diagram (hand-drawn or using a tool like draw.io) for your chosen feature, including at least: a load balancer, two app servers, a database, and a storage service (like AWS S3 or Azure Blob). Label each component clearly.**

   Ans :


Users
Load Balancer
App Server 1
AI Processing
User Requests
App Server 2
Recommendation
Music Services
Database
User Profiles
Preferences
Listening Data



- Component Description
- Users
    - Spotify users access the AI-Based Personalized Music Room feature through the mobile or web application.
- Load Balancer
    - Distributes incoming user requests across multiple application servers.
    - Ensures high availability and prevents a single server from becoming overloaded.
- App Server 1
    - Handles user requests.
    - Processes AI-based playlist generation and personalization logic.
- App Server 2
    - Handles additional traffic.
    - Provides recommendation services and improves reliability through redundancy.
- Database

    - Stores user profiles, listening history, preferences, and recommendation data.
    - Example: Amazon RDS or Azure SQL Database.
- Storage Service
    - Stores large files such as music metadata, AI model files, and other application assets.
    - Example: Amazon S3 or Azure Blob Storage.

3. **List three security measures you would add to your architecture to protect user data and prevent attacks. For each, explain in 1-2 lines how it helps (e.g., HTTPS, firewalls, authentication methods).**

   Ans :

   - HTTPS (TLS Encryption): Encrypts data transmitted between users and the system, preventing attackers from intercepting sensitive information such as passwords or personal details.
   - Authentication and Access Control: Uses methods like multi-factor authentication (MFA) and role-based permissions to ensure only authorized users can access protected data and features.
   - Firewalls and Intrusion Detection Systems (IDS): Monitors and filters incoming network traffic to block malicious requests and detect potential attacks before they can harm the system.

4. **Write a short document (200-300 words) explaining how your architecture would handle a sudden spike in users during a big event (like IPL finals for BookMyShow or a major album release on Spotify). Describe which parts would scale and how.**

   Ans :

   During a major event such as an IPL final ticket sale or a popular album release, the system must handle a sudden increase in users

without slowing down or crashing. The architecture would use cloud-based scaling methods to automatically adjust resources based on demand.

The first component that would scale is the application servers. A load balancer would distribute incoming user requests across multiple servers, and additional servers would be created automatically when traffic increases. This prevents any single server from becoming overloaded and keeps response times fast.

The database layer would also be designed for high traffic. Read replicas and database caching would be used to handle a large number of user requests, such as checking availability, viewing content, or loading user profiles. Frequently accessed data would be stored in a cache to reduce pressure on the main database.

A Content Delivery Network (CDN) would help deliver static content such as images, videos, and music files from locations closer to users. This reduces latency and improves performance during high-demand periods.

The system would also include queue management for tasks that do not need immediate processing, such as sending notifications, processing payments, or updating recommendations. This prevents sudden spikes from overwhelming backend services.

Monitoring tools would continuously track server health, traffic patterns, and errors. Alerts would help the operations team respond quickly if any service experiences problems.

By combining auto-scaling, load balancing, caching, CDNs, and monitoring, the architecture can handle millions of users during peak events while maintaining reliability, speed, and a smooth user experience.

5. **Present your architecture design to a friend or family member, ask them for one piece of feedback or a question, and write down what they said along with your response.

Hint: Focus on explaining your choices simply, as if you were talking to someone new to cloud computing.**

   Ans :

   I explained my architecture design to a friend/family member using simple examples. I described how the system uses cloud services, multiple servers, load balancing, databases, caching, and security measures to keep the application fast and reliable even when many users access it at the same time.

   Feedback/Question from them:
   "How does the system avoid crashing when millions of people use it at the same time?"

   My response:
   I explained that the system does not depend on just one server. Instead, it uses multiple servers that share the workload through a load balancer. When more users join suddenly, such as during a big event, the cloud can automatically add more servers to handle the extra traffic. Caching and database scaling also help by reducing the amount of work each part of the system needs to do. This allows the application to continue running smoothly even during very high demand.

   Their feedback helped me understand that technical concepts like cloud scaling should be explained with simple real-world examples. For example, I compared it to opening more checkout counters in a busy store when more customers arrive, instead of making everyone wait in one long line.